■ 30-Day Python Course

# Day 1

## Environment Setup

Complete Instructor Documentation

| | | | |
|---|---|---|---|
| ■ | ■ | ■ | ■ |
| **1 Hour** | **6 Steps** | **3 Tools** | **Phase 1** |
| Session Length | Setup Steps | To Install | of 5 Phases |

# Session Overview

**Session Goal**
Get your full Python development environment set up and running — from zero to executing your first Python script — all within one hour.

**By End of Session**
Students will have Python 3.12, uv, and VS Code installed with all extensions, a working virtual environment, and a confirmed Hello World output.

## Session Timeline at a Glance

| Time | Step | Topic | Duration |
|---|---|---|---|
| 0:00 – 0:10 | `Step 1` | Install Python 3.12 | ~10 min |
| 0:10 – 0:20 | `Step 2` | Install uv Package Manager | ~10 min |
| 0:20 – 0:30 | `Step 3` | Install VS Code | ~10 min |
| 0:30 – 0:38 | `Step 4` | Install VS Code Extensions | ~8 min |
| 0:38 – 0:50 | `Step 5` | Create Virtual Environment | ~12 min |
| 0:50 – 1:00 | `Step 6` | Hello World Script | ~10 min |

**■ Instructor Note**

This session is purely setup — no Python concepts are taught. Keep energy high by explaining WHY each tool is used. Students often get stuck on PATH issues on Windows; budget an extra 5 minutes for that OS specifically.

# Prerequisites & Preparation

## What Students Need Before Starting

| Requirement | Details |
|---|---|
| **Computer** | Windows 10/11, macOS 12+, or Ubuntu 22.04+. Any modern laptop works. |
| **Internet Connection** | Required for all downloads. ~500 MB total. |
| **Terminal Access** | Command Prompt / PowerShell (Windows) or Terminal (Mac/Linux) |
| **Admin Rights** | Needed to install software. Ensure they can install apps. |
| **Disk Space** | At least 2 GB free for Python, VS Code, and extensions |
| **Prior Knowledge** | None. This is Day 1 — complete beginner friendly. |

## Instructor Pre-Session Checklist

- ■ Test all installation commands on the target OS the night before
- ■ Have the Python 3.12 download page (python.org/downloads) bookmarked
- ■ Have the uv install page (astral.sh/uv) ready to share
- ■ Have VS Code download page (code.visualstudio.com) bookmarked
- ■ Test the Hello World script runs cleanly on your own machine
- ■ Prepare a fallback: have Python 3.12 installer downloaded locally if WiFi is slow
- ■ Know the Windows PATH fix: System Properties → Environment Variables → Path

# Step-by-Step Instruction Guide

**■ Format**

Each step includes: a script for what to say on camera, exact commands to type, common errors to watch for, and a verification checkpoint.

## STEP 1 — Install Python 3.12 (0:00 – 0:10)

### What to Say

"Python 3.12 is the version we're using for the entire course. Your system may have Python 2 or an older 3.x, but we want 3.12 specifically — it has the match statement, improved error messages, and performance improvements we'll use later."

### macOS Instructions

```
# Check if Homebrew is installed
$ brew --version

# If missing, install Homebrew first:
$ /bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# Install Python 3.12
$ brew install python@3.12

# Verify
$ python3.12 --version
> Python 3.12.x
```

### Windows Instructions

Go to **python.org/downloads** → click the Python 3.12.x button → run the installer.

**■■ CRITICAL — Windows Only**

On the very first installer screen, tick: "Add Python 3.12 to PATH". Missing this is the #1 Day 1 mistake on Windows. Python won't be found in the terminal without it.

```
# Verify in Command Prompt or PowerShell
$ python --version
> Python 3.12.x
```

```
# If 'python' doesn't work, try:
$ py --version
> Python 3.12.x
```

### Linux (Ubuntu/Debian) Instructions

```
$ sudo apt update
$ sudo apt install python3.12 python3.12-venv python3.12-dev

# Verify
$ python3.12 --version
> Python 3.12.x
```

### Common Errors — Step 1

| Error | Fix |
|---|---|
| `python: command not found` | On Mac/Linux use python3.12. On Windows re-run installer and tick PATH checkbox. |
| `Python 3.9 or older shows up` | Multiple Pythons installed. Use python3.12 explicitly. |
| `'brew' not found on Mac` | Homebrew not installed. Run the brew one-liner above first. |
| `Permission denied on Linux` | Add sudo before the apt commands. |

**■ Step 1 Done When...**

Every student runs python3.12 --version and sees Python 3.12.x printed in their terminal.

## STEP 2 — Install uv (0:10 – 0:20)

### What to Say

"In the old days, Python developers used pip to install packages and venv to create virtual environments — three separate tools. uv replaces all of them. It's written in Rust, 100x faster than pip, and handles environments and packages in one command. Every project in this course uses uv."

### macOS / Linux

```
$ curl -LsSf https://astral.sh/uv/install.sh | sh

# Restart terminal OR source your shell config:
```

```
$ source ~/.zshrc # macOS (zsh default)
$ source ~/.bashrc # Linux

# Verify
$ uv --version
> uv 0.x.x
```

### Windows (PowerShell — run as Administrator)

```
$ powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"

# Close and reopen PowerShell, then verify:
$ uv --version
> uv 0.x.x
```

### Common Errors — Step 2

- 'uv' is not recognised — Close the terminal fully and reopen. The PATH needs refreshing.
- curl not found on Windows — Use the PowerShell command instead.
- SSL certificate error — Corporate firewall. Try curl -k or download manually from github.com/astral-sh/uv/releases.

**■ Step 2 Done When...**

uv --version returns a version string in every student's terminal.

## STEP 3 — Install VS Code (0:20 – 0:30)

### What to Say

"VS Code is a free, open-source editor from Microsoft and the most popular editor for Python in the world. We'll use it for all 30 days — writing code, debugging, running tests, and managing Git. With the right extensions it's just as powerful as PyCharm but much faster."

### Install

Go to **code.visualstudio.com** → Download for your OS → Run the installer. Accept all defaults.

```
# macOS: enable the 'code' terminal command after installing
# 1. Open VS Code
# 2. Press Cmd+Shift+P
# 3. Type: Shell Command: Install 'code' command in PATH
```

```
# 4. Press Enter

# Verify from terminal:
$ code --version
> 1.xx.x
```

**■ Tip**

On Windows, the installer offers to 'Add to PATH' and 'Add Open with Code to context menu' — tick both. These save time throughout the course.

**■ Step 3 Done When...**

VS Code opens successfully AND code --version works in terminal (needed for opening project folders from the command line).

## STEP 4 — Install VS Code Extensions (0:30 – 0:38)

### What to Say

"VS Code is bare-bones out of the box. These four extensions turn it into a fully-equipped Python environment. Each one has a specific role — I'll explain as we install."

### Extension Details

| Extension | Publisher | What It Does |
|---|---|---|
| **Python** | Microsoft | Debugger, test runner, IntelliSense. The core extension — without this VS Code knows nothing about Python. |
| **Pylance** | Microsoft | Type-aware autocomplete and real-time error detection. Works with the Python extension to give IDE-level analysis. |
| **Ruff** | Astral | Linter and formatter in one — replaces both flake8 and black. Formats your code automatically on save. |
| **GitLens** | GitKraken | Inline git blame, visual history, branch comparison. Essential from Day 26 onwards. |

### How to Install

- Press Ctrl+Shift+X (Windows/Linux) or Cmd+Shift+X (Mac) to open Extensions panel
- Search for each extension by name
- Click Install — they install in seconds
- Verify: you see a blue Disable / Uninstall option, meaning it's active

**■ Step 4 Done When...**

All four extensions show as Installed in the Extensions panel and no error notifications appear in VS Code.

## STEP 5 — Create Virtual Environment (0:38 – 0:50)

### What to Say

"A virtual environment is an isolated Python installation just for your project. When you install a package inside it, it only exists here — not globally. Different projects can use different package versions without conflicting. Every Python project in this course gets its own environment. With uv, it takes two seconds."

### Commands

```
# 1. Create a folder for all course work
$ mkdir python-course && cd python-course

# 2. Initialise a uv project
$ uv init

# 3. Create the virtual environment
$ uv venv
> Using CPython 3.12.x interpreter at ...
> Creating virtual environment at: .venv

# 4. Activate it
$ source .venv/bin/activate # macOS / Linux
$ .venv\Scripts\Activate.ps1 # Windows PowerShell
$ .venv\Scripts\activate.bat # Windows Command Prompt

# You should now see (.venv) before your prompt:
(.venv) $
```

### Connect VS Code to the Environment

- Open the project folder: code .
- VS Code auto-detects .venv and shows a notification — click Yes
- Or: Cmd/Ctrl+Shift+P → Python: Select Interpreter → choose the .venv entry
- Check bottom status bar — it should show Python 3.12.x (.venv)

**■ Why .venv?**

uv names the folder .venv by convention (the dot makes it hidden on Mac/Linux). VS Code looks for .venv automatically, which is why it pops up a notification immediately.

### Common Errors — Step 5

- PowerShell 'running scripts is disabled' — Run: Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser then retry.
- (.venv) prefix doesn't appear — Close and reopen the terminal inside VS Code (Terminal → New Terminal).
- VS Code doesn't detect interpreter — Cmd+Shift+P → Python: Select Interpreter → Browse → navigate to .venv/bin/python3

**■ Step 5 Done When...**

Terminal shows (.venv) prefix AND VS Code status bar shows Python 3.12.x (.venv) in the bottom-left corner.

## STEP 6 — Hello World (0:50 – 1:00)

### What to Say

"This is the moment we verify the entire stack is working end-to-end. We'll write a real Python function — not just a print statement — so you get a preview of what Day 5 (Functions) looks like. Don't worry about understanding every line yet. Just get it running."

### Create hello.py in VS Code

Create a new file called **hello.py** in the python-course folder and type:

```
# hello.py — your first Python script

def greet(name: str) -> str:
return f"Hello, {name}! Welcome to Python 3.12."

if __name__ == "__main__":
message = greet("World")
print(message)
```

### Run It

```
# Make sure (.venv) is active, then:
$ python hello.py
> Hello, World! Welcome to Python 3.12.
```

### Line-by-Line Explanation

| Code | Explanation |
| --- | --- |
| `# hello.py` | Comment — Python ignores this line. Used for notes. |
| `def greet(name: str) -> str:` | Defines a function. name: str is a type hint (optional). -> str says it returns a string. Covered fully Day 5. |
| `return f"Hello, {name}!..."` | f-string: embeds the variable name directly into the string. Covered in detail on Day 7. |
| `if __name__ == "__main__":` | Guard clause: code here only runs when you execute this file directly, not when it's imported. You'll see this everywhere. |
| `message = greet("World")` | Calls the function with argument "World" and stores the return value. |
| `print(message)` | Outputs the message to the terminal. |

**■ Step 6 Done When...**

Every student sees "Hello, World! Welcome to Python 3.12." printed in their terminal with no errors. ■

# Session Wrap-Up

## End-of-Session Student Checklist

| |
|---|
| ■ Python 3.12 installed — python3.12 --version shows 3.12.x |
| ■ uv installed — uv --version returns a version string |
| ■ VS Code installed and opens correctly |
| ■ All 4 extensions installed: Python, Pylance, Ruff, GitLens |
| ■ python-course/ folder created with uv project initialised |
| ■ Virtual environment (.venv) created and activated — see (.venv) prefix |
| ■ VS Code connected to .venv — status bar shows Python 3.12.x (.venv) |
| ■ hello.py created and runs successfully — output matches exactly |

## Key Concepts to Recap (last 2 minutes)

- Python 3.12 is our version for all 30 days — always use this, not older versions
- uv replaces pip + venv — one tool, faster, modern
- Virtual environment = isolated sandbox per project. Always activate before working.
- VS Code + extensions = our IDE for the entire course
- Every project starts with: uv init → uv venv → source .venv/bin/activate → code .

## Preview of Day 2

**■ Tomorrow — Day 2: Variables & Data Types**

Students will write their first interactive programs using variables, data types (int, float, str, bool), type(), input(), print(), and type casting. Make sure they keep the python-course folder — Day 2 adds files to it.

## Instructor Troubleshooting Notes

### If a student's setup is broken at session end

- Don't hold the class back. Help them offline or in a breakout.
- Common fix: delete .venv folder and re-run uv venv from scratch.
- Nuclear option: uninstall Python, restart machine, reinstall from the exact steps above.

## Windows-specific notes

- python command may need to be py — document this for affected students.
- PowerShell execution policy: Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
- Broken PATH: search 'Edit system environment variables' → Environment Variables → User Path → add Python 3.12 Scripts folder manually.

## macOS-specific notes

- Apple Silicon (M1/M2/M3): Homebrew installs to /opt/homebrew. If brew commands fail: echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile
- System Python: macOS ships Python 2.7 or 3.x. Always use python3.12 explicitly.

## Linux-specific notes

- Some distros need python3.12-venv separately: sudo apt install python3.12-venv
- Permission errors on system directories: always use sudo for apt, never for pip inside .venv

# Quick Reference Card

Pin or print this page. These commands are used every single day of the course.

## Daily Workflow Commands

```
# Navigate to your project
$ cd python-course

# Activate environment (do this EVERY time you open a new terminal)
$ source .venv/bin/activate # macOS / Linux
$ .venv\Scripts\Activate.ps1 # Windows PowerShell

# Open VS Code in current folder
$ code .

# Run a Python file
$ python filename.py

# Install a package (with venv active)
$ uv add package-name

# Deactivate environment when done
$ deactivate
```

## VS Code Keyboard Shortcuts

| Mac | Windows / Linux | Action |
|---|---|---|
| `Cmd+Shift+P` | `Ctrl+Shift+P` | Command Palette — access all VS Code features |
| `Cmd+`` | `Ctrl+`` | Open / toggle integrated terminal |
| `Cmd+Shift+X` | `Ctrl+Shift+X` | Extensions panel |
| `Cmd+S` | `Ctrl+S` | Save file (auto-formats with Ruff on save) |
| `F5` | `F5` | Run file with debugger |
| `Cmd+/` | `Ctrl+/` | Toggle line comment |
| `Cmd+D` | `Ctrl+D` | Select next occurrence of current word |

## Tool Summary

| Tool | Version | Purpose |
|---|---|---|
| **Python** | 3.12.x | The language runtime. Executes all our code. |
| **uv** | Latest | Package manager + virtual environment tool. Replaces pip + venv. |
| **VS Code** | Latest | Our code editor for all 30 days. |
| **Python ext.** | Latest | VS Code integration: debugger, linter, test runner. |
| **Pylance** | Latest | Type-aware autocomplete and real-time error highlighting. |
| **Ruff** | Latest | Auto-formatter and linter. Runs on every file save. |
| **GitLens** | Latest | Git integration inside VS Code. Used from Day 26+. |